Project 17 (Real-World Example) — DeflectDesk-Support-Bot-With-Confidence-Gated-Human-Escalation

Type: RAG agent on LangGraph with human-in-the-loop · **Stack:** Python, LangChain, Chroma (RAG), LangGraph (`interrupt`, checkpointer), OpenAI SDK · **Level:** Intermediate

Tagline: A support bot that answers from your help docs when it's confident — and, crucially, *hands the conversation to a human* when it isn't, instead of making something up.

Charter (what this project is)

Most support bots fail the same way: they answer *everything*, confidently, including the questions they have no business answering. **DeflectDesk** doesn't. It searches your help docs for the user's question, and only answers when the best match is **close enough** (a distance below a cutoff). When nothing relevant is found, it does the one thing a bot almost never does — it **escalates to a human**, pausing mid-conversation (`interrupt`) until a person replies. The honest "let me get someone" is worth more than a confident wrong answer.

Jump to: Project Structure · Setup · How testing works · Step 1 · Step 2 · Step 3

The Story — what you're building

A question comes in. A **retrieve** node searches the help docs (Chroma) and records the best match's distance. A **router** checks it against a cutoff: close enough → the **answer** node replies grounded in that doc; too far → the **escalate** node calls `interrupt()`, which **pauses the whole graph** and waits. A human reads the paused question, types a reply, and the graph **resumes** with their answer. A checkpointer saves the state so the pause survives — the conversation can wait minutes or hours for a person.

The lesson: the valuable skill isn't answering — it's knowing **when not to**. A confidence gate (a retrieval distance under a cutoff) plus a real human handoff (`interrupt` + checkpointer) is what separates a support bot you can trust from one that invents refund policies. This is Doc08's RAG and Doc09/10's human-in-the-loop graph, pointed at the one decision that matters: answer, or get a human.

Project Structure (the professional layout)

`main.py` at the repo root runs a question; all code in `src/`.

```
<repo folder>/ (the DeflectDesk repo — full name in the heading above)
├── data/
│   └── help_docs/              a few short .md help articles
├── src/
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── rag/
│   │   ├── __init__.py
│   │   ├── help_store.py       build_help_store()        (Step 1)
│   │   └── retriever.py        best_match()              (Step 1)
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── config.py
│   ├── confidence.py           is_confident()            (Step 1)
│   ├── exceptions.py
│   ├── graph.py                build_graph()             (Step 2)
│   └── state.py                SupportState, start_state (Step 2)
├── tests/
│   ├── integration/test_live_support.py
│   └── unit/                   test_config, test_confidence
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
├── README.md
└── requirements.txt
```

How the structure grows, step by step

- **Step 1** builds the standard setup, `src/rag/` (help store + retriever) and `src/confidence.py` (the deterministic gate) with tests — the gate tested with no model.
- **Step 2** adds `src/state.py` and `src/graph.py` (retrieve → route → answer/escalate, with `interrupt`).
- **Step 3** adds the checkpointer, the pause/resume flow, `main.py`, and the integration test.

Setup (do this once, before Step 1)

```
cd DeflectDesk-Support-Bot-With-Confidence-Gated-Human-Escalation
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install langchain-openai langchain-community chromadb langgraph \
    langgraph-checkpoint-sqlite openai python-dotenv
pip freeze > requirements.txt
```

`.env.example` (copy to `.env`): `OPENAI_API_KEY=sk-...`, `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `chroma_db/`, `*.sqlite`. Every folder under `src/` gets an empty `__init__.py`. Write 3-4 short help articles in `data/help_docs/` (refunds, password reset, shipping) and leave a clearly off-topic question (like "delete my account") with no article, so escalation has something to trigger on. `LICENSE` is the MIT text.

How testing works in this project

Two layers, one style (plain OK/WRONG scripts; this builds on Doc08/09, not pytest):

- **`tests/unit/`** — no model: `test_config`; `test_confidence` (the gate — a small distance is confident, a large one is not, exactly at the cutoff is confident).
- **`tests/integration/test_live_support.py`** — the real model + RAG + graph: a question covered by the docs is answered from them; an off-topic question escalates (the graph pauses), and resuming with a human reply finishes with `escalated = True`.

Each step ends with a **Tests for this step** box.

Step 1 — The Help-Docs RAG and the Confidence Gate

What this step does: the help-docs store, the retriever, and the deterministic **confidence gate** — the one decision the whole bot turns on, testable with no model.

Why this step matters: "answer or escalate" is entirely decided by whether the best match's distance is under the cutoff. Prove that gate with made-up distances, and the graph in Step 2 only has to route on it.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write the standard setup (`config` / `exceptions` / `utils/logger` / `models/llm`). 2. Write `src/rag/help_store.py`: `build_help_store()` — load the help articles, embed them (`OpenAIEmbeddings`), add to a Chroma collection. `src/rag/retriever.py`: `best_match(collection, question)` → `(text, distance)` of the closest article (a big distance if empty). 3. Write `src/confidence.py`: `is_confident(distance, cutoff) -> bool` — `distance <= cutoff`. This is the gate.

Tests for this step: `test_confidence` — a small distance is confident, a large one is not, the cutoff value itself is confident. `test_config`.

Step 2 — The Graph: Answer or Escalate

What this step does: the LangGraph graph — a retrieve node, a conditional route on the gate, an answer node (grounded in the matched doc), and an escalate node that calls `interrupt()`.

Why this step matters: this is the whole behavior as explicit graph structure — the route *is* the answer-or-escalate decision, and `interrupt()` is the real human handoff.

What's new vs. Step 1: `src/state.py`, `src/graph.py`. **What stays the same:** the RAG and the gate.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `src/state.py`: `SupportState` (question, context, best_distance, answer, escalated) and `start_state(question)`. 2. Write `src/graph.py`: `retrieve_node` (fills context + best_distance via `best_match`), a `route` function (`is_confident` → "answer" else "escalate"), `answer_node` (ask the model to answer *only* from the context), `escalate_node` (`human = interrupt({...}); return {"answer": human, "escalated": True}`), wired `START → retrieve → {answer, escalate} → END`.

Tests for this step: proven live in Step 3; for now, a confident state routes to "answer" and a non-confident one to "escalate".

Step 3 — Pause, Resume, and the CLI (final)

What this step does: compile the graph with a checkpointer, run it, and handle the pause — when it escalates, the run stops at the escalate node; a human provides an answer and the graph resumes.

Why this step matters: the checkpointer is what makes the pause *real* — the conversation's state is saved, so a human can reply minutes later and the graph picks up exactly where it stopped.

What's new vs. Step 2: the checkpointer, `main.py`, the integration test. **What stays the same:** the graph.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Compile with a checkpointer (`MemorySaver` for a demo; `SqliteSaver` to survive a restart). Run `graph.invoke(start_state(q), config)`; after it, `graph.get_state(config).next` is non-empty when it paused at the escalate node. 2. Resume with `graph.invoke(Command(resume=human_answer), config)`. 3. Write `main.py` (answer a

question; if it escalates, prompt for a human reply and resume) and `tests/integration/test_live_support.py`.

Final Deliverable: a support bot that answers from your docs when confident and hands off to a human — pausing and resuming — when it isn't.

Tests for this step: `test_live_support` — a covered question is answered from the docs; an off-topic one escalates and, on resume, finishes with `escalated = True`.

Run, Test and Configure the Final Project

```
python main.py "How do refunds work?"
python main.py "Delete my account permanently"
python -m tests.unit.test_config
python -m tests.unit.test_confidence
python -m tests.integration.test_live_support
```

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	Log verbosity
`CONFIDENCE_CUTOFF`	no	`0.8`	Max distance that still counts as confident

Checklist Before You Call This Done

- [ ] The bot answers only when the best match is under the cutoff
- [ ] An off-topic question escalates — it does not invent an answer
- [ ] `is_confident` is a deterministic gate, tested with no model
- [ ] Escalation really pauses the graph (`interrupt`) and resumes with a human reply
- [ ] A checkpointer saves the state, so the pause survives

Publish It to GitHub

Name the repo `DeflectDesk-Support-Bot-With-Confidence-Gated-Human-Escalation`, add the `README.md`, push. "A bot that knows when to get a human" is a mature, trustworthy demo that stands out.

Status

Not started.

Stuck? Ask for **Hint 1** or **Hint 2** before the solution. Build the confidence gate (Step 1) first — it's the one decision everything else routes on.